

HematoVision - Advanced Blood Cell Classification

Project Description

HematoVision is a Deep Learning based automated blood cell classification system developed to identify microscopic white blood cells using Transfer Learning. The model classifies images into EOSINOPHIL, LYMPHOCYTE, MONOCYTE, and NEUTROPHIL categories.

Images were resized to 224x224 and preprocessed using MobileNetV2 preprocess_input. The dataset was internally split into train, validation and test sets.

Training Configuration:

- Epochs: 5
- Batch Size: 8
- Image Size: 224x224
- Base Model: Fine-tuned

Performance Achieved:

- Training Accuracy: 79.15%
- Validation Accuracy: 87.60%
- Test Accuracy: 87.28%

Project Flow

- 1 Dataset Collection from Kaggle
- 2 DataFrame creation with image paths and labels
- 3 Dataset splitting (Train / Validation / Test)
- 4 Image preprocessing using MobileNetV2 preprocess_input
- 5 Model Building using Transfer Learning
- 6 Model Training (5 epochs)
- 7 Model Evaluation using Confusion Matrix and Accuracy
- 8 Deployment using Flask Web Application
- 9 Real-time Prediction using model.predict()

Milestone 1: Data Collection

Activity 1: Downloading Dataset

```
import os

dataset_path = "/content/blood_cell_images/TRAIN"
for folder in os.listdir(dataset_path):
```

```
print(folder)
```

Activity 2: Creating DataFrame

```
import pandas as pd

filepaths = []
labels = []

for folder in os.listdir(dataset_path):
    folder_path = os.path.join(dataset_path, folder)
    for file in os.listdir(folder_path):
        filepaths.append(os.path.join(folder_path, file))
        labels.append(folder)

bloodCell_df = pd.DataFrame({
    "filepaths": filepaths,
    "labels": labels
})
```

Milestone 2: Data Preprocessing

Activity 1: Dataset Splitting

```
from sklearn.model_selection import train_test_split

train_images, test_images = train_test_split(
    bloodCell_df,
    test_size=0.3,
    stratify=bloodCell_df["labels"],
    random_state=42
)
```

Activity 2: Image Preprocessing

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import tensorflow as tf

image_gen = ImageDataGenerator(
    preprocessing_function=tf.keras.applications.mobilenet_v2.preprocess_input
)
```

Milestone 3: Model Building

Activity 1: Load MobileNetV2

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
```

```
        weights="imagenet"  
    )
```

Activity 2: Add Layers

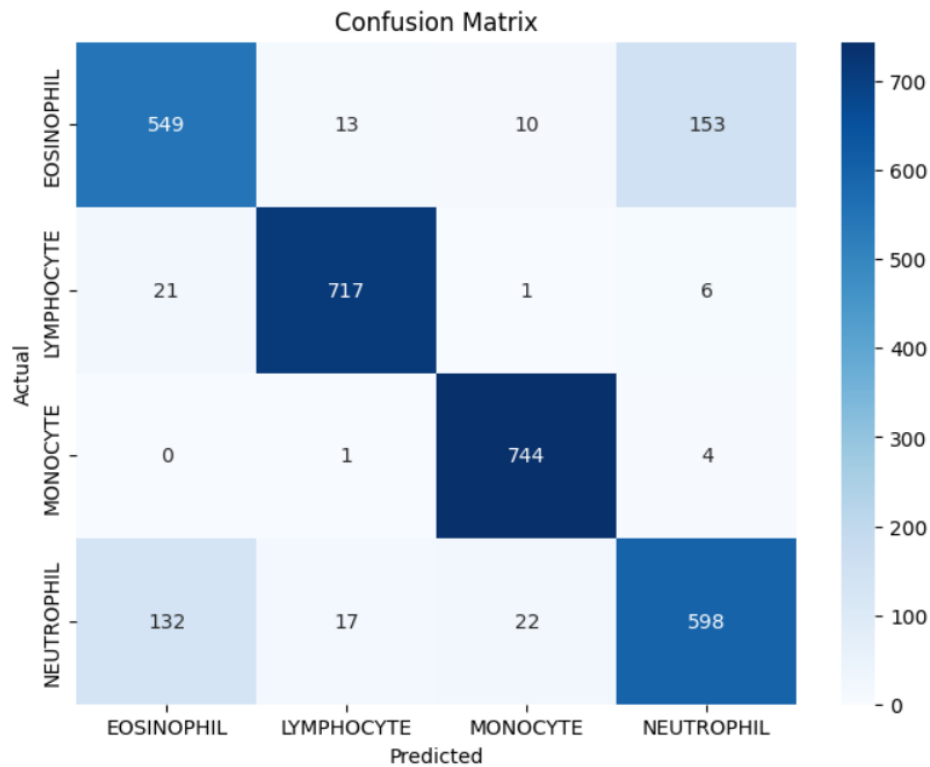
```
from tensorflow.keras import layers, models  
  
x = base_model.output  
x = layers.GlobalAveragePooling2D()(x)  
output = layers.Dense(4, activation="softmax")(x)  
model = models.Model(inputs=base_model.input, outputs=output)
```

Activity 3: Compile & Train

```
model.compile(optimizer="adam",  
              loss="categorical_crossentropy",  
              metrics=["accuracy"])  
  
history = model.fit(train, epochs=5, validation_data=val)
```

Milestone 4: Model Evaluation

```
from sklearn.metrics import confusion_matrix, classification_report  
  
pred = model.predict(test)  
pred_classes = pred.argmax(axis=1)  
  
cm = confusion_matrix(test.classes, pred_classes)  
print(classification_report(test.classes, pred_classes))
```



Milestone 5: Application Building

```
from flask import Flask, render_template, request
from tensorflow.keras.models import load_model
import numpy as np
import cv2

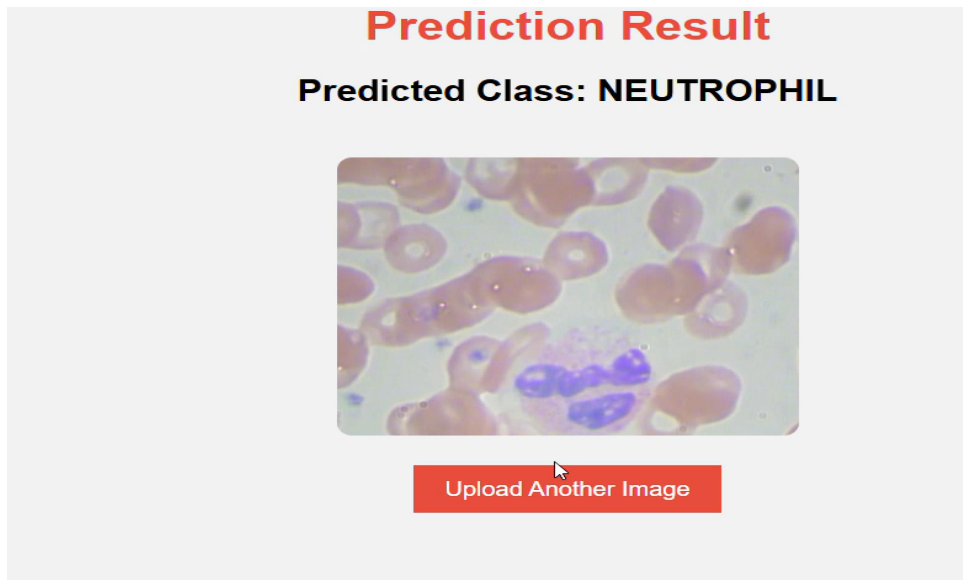
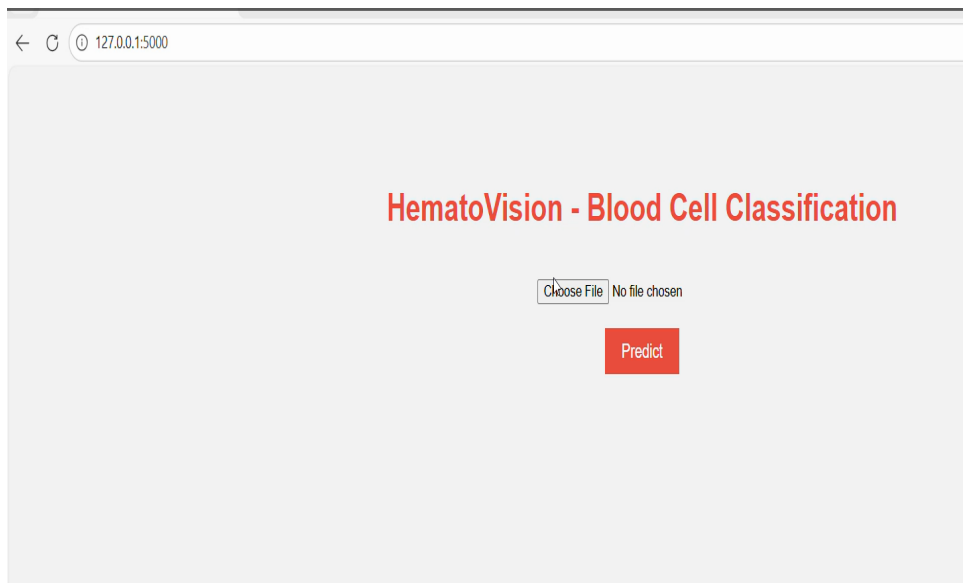
app = Flask(__name__)
model = load_model("Blood Cell.h5")

def predict_image(img_path):
    img = cv2.imread(img_path)
    img = cv2.resize(img, (224, 224))
    img = tf.keras.applications.mobilenet_v2.preprocess_input(img)
    img = np.expand_dims(img, axis=0)
    prediction = model.predict(img)
    return np.argmax(prediction)
```

Results & Performance Analysis

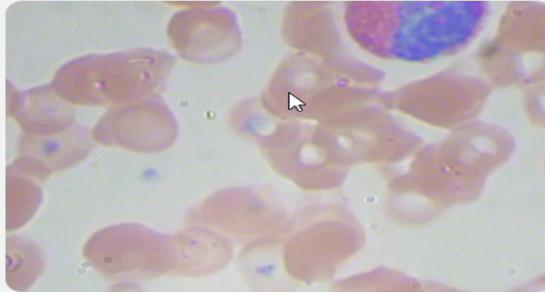
Validation Accuracy: 87.60%
 Test Accuracy: 87.28%

The Flask web application successfully predicts the blood cell class for uploaded microscopic images.



Prediction Result

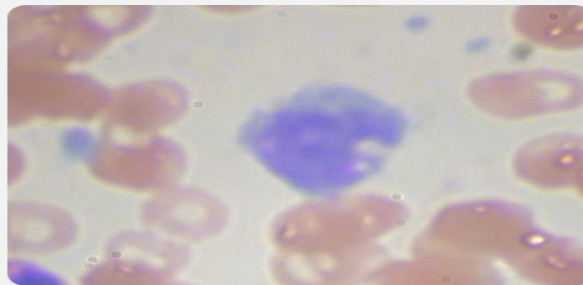
Predicted Class: EOSINOPHIL



[Upload Another Image](#)

Prediction Result

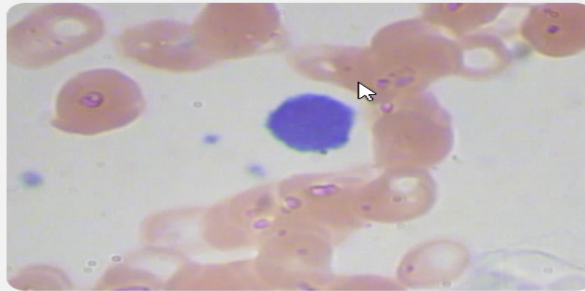
Predicted Class: MONOCYTE



[Upload Another Image](#)

Prediction Result

Predicted Class: LYMPHOCYTE



[Upload Another Image](#)